Case Study: Android Stack Architecture

Date Of Performance:

Date Of Submission:

AIM: To demonstrate Android Stack Architecture.

Theory:
Android is an open source, Linux-based software stack created for a wide array of devices and form factors.

Linux kernel

The foundation of the Android platform is the Linux kernel. For example, the Android Runtime (ART) relies on the Linux kernel for underlying functionalities such as threading and low-level memory management.

Using a Linux kernel lets Android take advantage of key security features and lets device manufacturers develop hardware drivers for a well-known kernel.

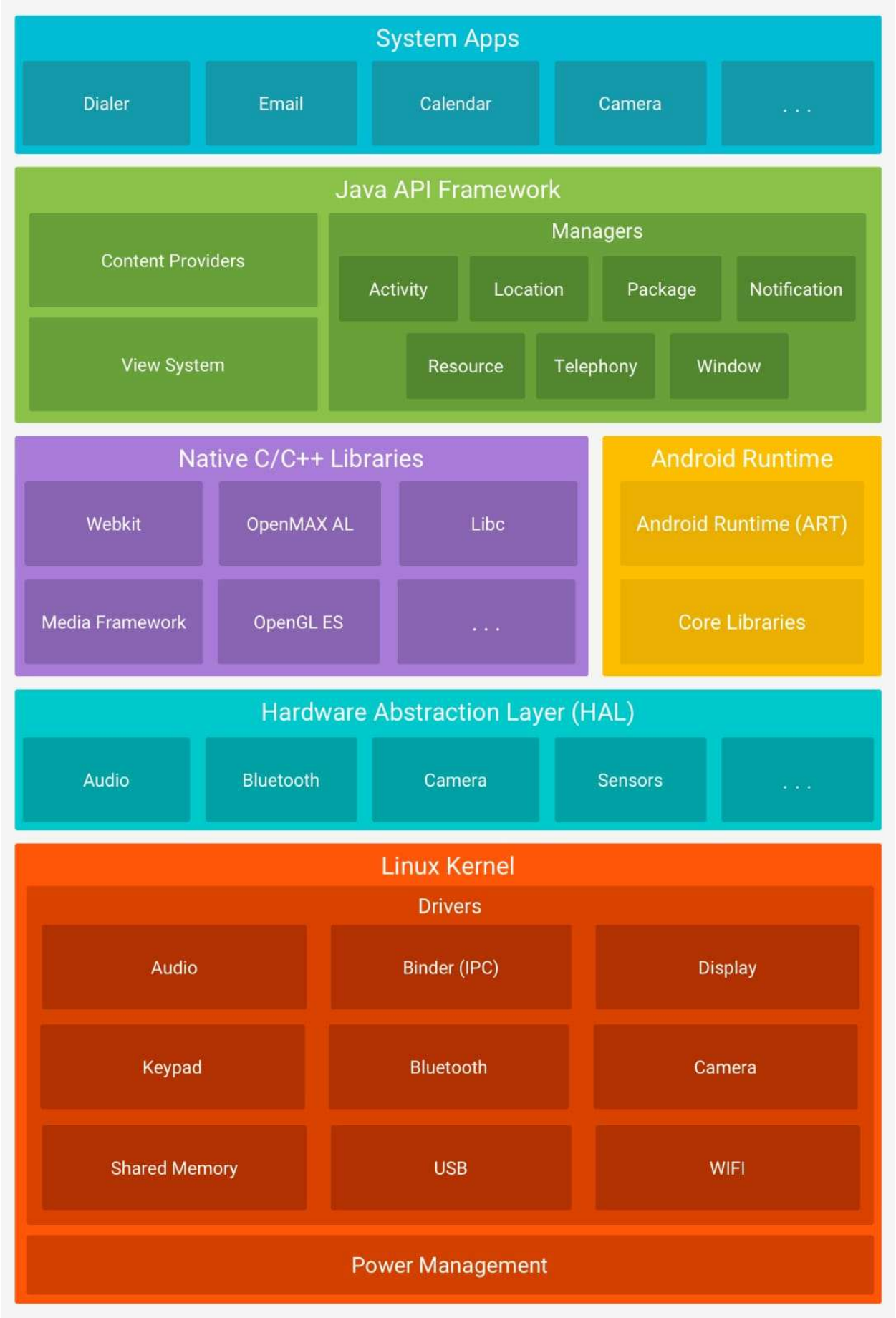
Android Libraries
This category encompasses those Java-based libraries that are specific to Android development. Examples of libraries in this category include the application framework libraries in addition to those that facilitate user interface building, graphics drawing and database access. A summary of some key core Android libraries available to the Android developer is as follows −

Having covered the Java-based core libraries in the Android runtime, it is now time to turn our attention to the C/C++ based libraries contained in this layer of the Android software stack.

● android.app − Provides access to the application model and is the cornerstone of all Android applications.
● android.content − Facilitates content access, publishing and messaging between applications and application components.
● android.database − Used to access data published by content providers and includes SQLite database management classes.
● android.opengl − A Java interface to the OpenGL ES 3D graphics rendering API.
● android.os − Provides applications with access to standard operating system services including messages, system services and inter-process communication.3
● android.text − Used to render and manipulate text on a device display.
● android.view − The fundamental building blocks of application user interfaces.
● android.widget − A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
● android.webkit − A set of classes intended to allow web-browsing capabilities to be built into applications.

Hardware abstraction layer (HAL)

The hardware abstraction layer (HAL) provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework. The HAL consists of multiple library modules, each of which implements an interface for a specific type of hardware component, such as the camera or Bluetooth module. When a framework API makes a call to access device hardware, the Android system loads the library module for that hardware component.


System Apps
Dialer
Email
Calendar
Camera
. . .
Java API Framework
Content Providers
View System
Managers
Activity
Location
Package
Notification
Resource
Telephony
Window
Native C/C++ Libraries
Webkit
OpenMAX AL
Libc
Media Framework
OpenGL ES
. . .
Android Runtime
Android Runtime (ART)
Core Libraries
Hardware Abstraction Layer (HAL)
Audio
Bluetooth
Camera
Sensors
. . .
Linux Kernel
Drivers
Audio
Binder (IPC)
Display
Keypad
Bluetooth
Camera
Shared Memory
USB
WIFI
Power Management

Android runtime

For devices running Android version 5.0 (API level 21) or higher, each app runs in its own process and with its own instance of the Android Runtime (ART). ART is written to run multiple virtual machines on low-memory devices by executing Dalvik Executable format (DEX) files, a bytecode format designed specifically for Android that's optimized for a minimal memory footprint. Build tools, such as d8, compile Java sources into DEX bytecode, which can run on the Android platform.

Some of the major features of ART include the following:

●Ahead-of-time (AOT) and just-in-time (JIT) compilation

●Optimized garbage collection (GC)

●On Android 9 (API level 28) and higher, conversion of an app package's DEX files to more compact machine code

●Better debugging support, including a dedicated sampling profiler, detailed diagnostic exceptions and crash reporting, and the ability to set watchpoints to monitor specific fields

Prior to Android version 5.0 (API level 21), Dalvik was the Android runtime. If your app runs well on ART, then it can work on Dalvik as well, but the reverse might not be true.

Android also includes a set of core runtime libraries that provide most of the functionality of the Java programming language, including some Java 8 language features, that the Java API framework uses.

Native C/C++ libraries

Many core Android system components and services, such as ART and HAL, are built from native code that requires native libraries written in C and C++. The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to apps. For example, you can access OpenGL ES through the Android framework's Java OpenGL API to add support for drawing and manipulating 2D and 3D graphics in your app.

If you are developing an app that requires C or C++ code, you can use the Android NDK to access some of these native platform libraries directly from your native code.

Java API framework

The entire feature-set of the Android OS is available to you through APIs written in the Java language. These APIs form the building blocks you need to create Android apps by simplifying the reuse of core, modular system components and services, which include the following:

- A rich and extensible view system you can use to build an app's UI, including lists, grids, text boxes, buttons, and even an embeddable web browser
- A resource manager, providing access to non-code resources such as localized strings, graphics, and layout files
- A notification manager that enables all apps to display custom alerts in the status bar
- An activity manager that manages the lifecycle of apps and provides a common navigation back stack
- Content providers that enable apps to access data from other apps, such as the Contacts app, or to share their own data

Developers have full access to the same framework APIs that Android system apps use.

System apps

Android comes with a set of core apps for email, SMS messaging, calendars, internet browsing, contacts, and more. Apps included with the platform have no special status among the apps the user chooses to install. So, a third-party app can become the user's default web browser, SMS messenger, or even the default keyboard. Some exceptions apply, such as the system's Settings app.

The system apps function both as apps for users and to provide key capabilities that developers can access from their own app. For example, if you want your app to deliver SMS messages, you don't need to build that functionality yourself. You can instead invoke whichever SMS app is already installed to deliver a message to the recipient you specify.

Application Framework

The Application Framework layer provides many higher-level services to applications in the form of Java classes. Application developers are allowed to make use of these services in their applications.

The Android framework includes the following key services −

- Activity Manager − Controls all aspects of the application lifecycle and activity stack.
- Content Providers − Allows applications to publish and share data with other applications.
- Resource Manager − Provides access to non-code embedded resources such as strings, color settings and user interface layouts.
- Notifications Manager − Allows applications to display alerts and notifications to the user.
- View System − An extensible set of views used to create application user interfaces.

Android Stack Commands:

1)adb devices: This command is used to check whether your Android device is properly connected to your system.

Syntax: adb devices

Example:

```
adb devices
List of devices attached
emulator-5554 device
```

Purpose: It verify connection between PC and Android device/emulator before running any other command.

2)adb install: Installs an APK file onto a connected Android device.

Syntax: adb install <apk_file_path>

Example: adb install app-debug.apk

Purpose: It used during development to directly install apps without using Play Store.

3)adb uninstall: Removes an app from the device.

Syntax: adb uninstall <package_name>

Example: adb uninstall com.example.myapp

Purpose: It helps in testing clean installs or removing unwanted apps.

4)adb logcat: Displays real-time logs from the Android system.

Syntax: adb logcat

Example: adb logcat | findstr "MainActivity"

Purpose: Very useful for debugging crashes and checking system messages.

5)adb shell: Opens a command-line shell on the Android device.

Syntax: adb shell

Example: adb shell ls /sdcard/

Purpose: It allows direct interaction with Android's Linux-based system.

6)adb push: Copies files from your computer to the Android device.

Syntax: adb push <local_path> <device_path>

Example: adb push file.txt /sdcard/

Purpose: It used to transfer files like images, logs, or test data to the device.

7)adb pull: Copies files from the Android device to your computer.

Syntax: adb pull <device_path> <local_path>

Example: adb pull /sdcard/file.txt

Purpose: It helpful for retrieving logs or stored files from the device.

8)adb reboot: Restarts the Android device.

Syntax: adb reboot

Purpose: It used when testing system-level changes or debugging boot issues.

9)adb start-server/adb kill-server: Starts or stops the ADB server.

Syntax: adb start-server
adb kill-server

Purpose: It fixes connection issues when ADB is not responding properly.

10)adb shell dumpsys: Provides detailed system information about services.

Sign and Remarks:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature